

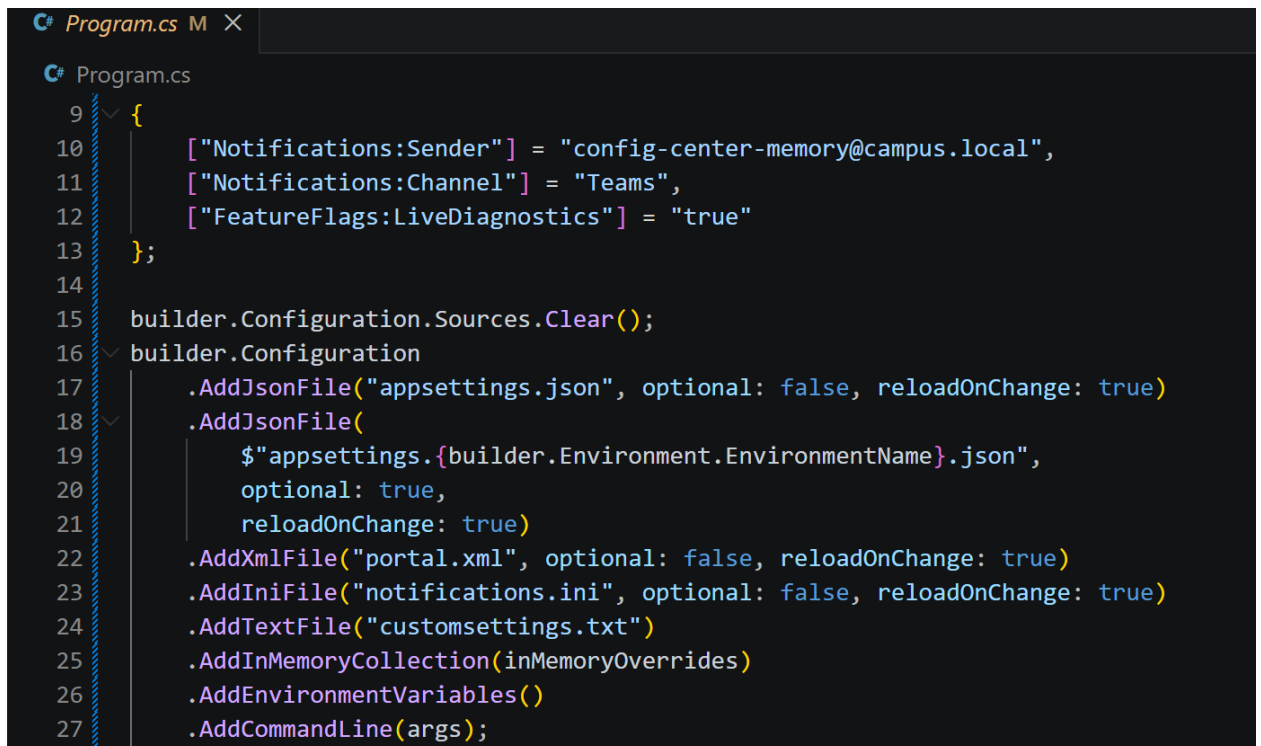
Отчёт о выполнении практической работы №6

1. Цель работы

Освоить сборку конфигурации ASP.NET Core из нескольких источников, анализ секций, создание собственного провайдера, привязку данных к классам (Get<T>()/Bind()) и передачу настроек через IOptions<T> в endpoints и custom middleware.

2. Порядок подключения источников конфигурации

В Program.cs используется явное очищение коллекции Sources для детерминированного порядка. Источники подключаются снизу вверх (последний подключённый имеет наивысший приоритет):

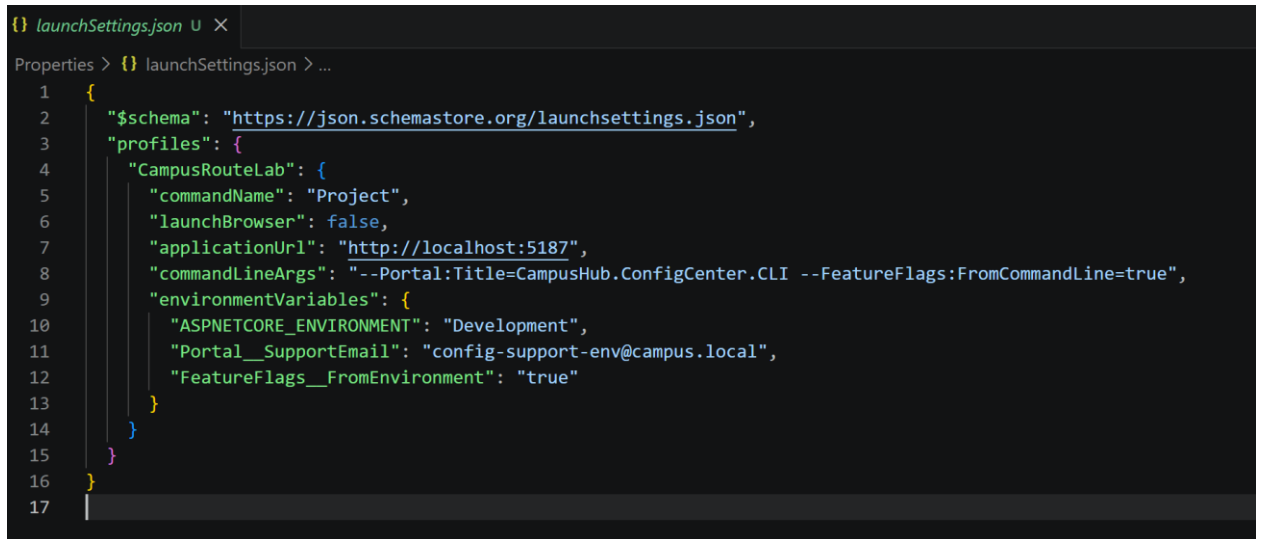


```
Program.cs M X
Program.cs
9  {
10     ["Notifications:Sender"] = "config-center-memory@campus.local",
11     ["Notifications:Channel"] = "Teams",
12     ["FeatureFlags:LiveDiagnostics"] = "true"
13 };
14
15 builder.Configuration.Sources.Clear();
16 builder.Configuration
17     .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
18     .AddJsonFile(
19         $"appsettings.{builder.Environment.EnvironmentName}.json",
20         optional: true,
21         reloadOnChange: true)
22     .AddXmlFile("portal.xml", optional: false, reloadOnChange: true)
23     .AddIniFile("notifications.ini", optional: false, reloadOnChange: true)
24     .AddTextFile("customsettings.txt")
25     .AddInMemoryCollection(inMemoryOverrides)
26     .AddEnvironmentVariables()
27     .AddCommandLine(args);
```

Принцип объединения: ASP.NET Core использует цепочку провайдеров. При запросе ключа система опрашивает их в порядке подключения. Значение из провайдера, подключённого позже, перезаписывает предыдущее. Это гарантирует предсказуемое переопределение настроек для разных окружений.

3. Параметры запуска (launchSettings.json)

Для профиля CampusRouteLab заданы переменные окружения и аргументы командной строки:

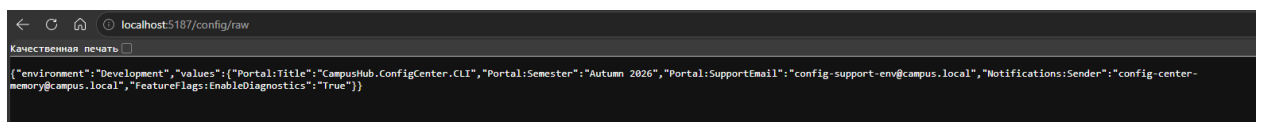
A screenshot of a code editor showing the launchSettings.json file. The file is a JSON object with a 'profiles' section containing a 'CampusRouteLab' profile. This profile has properties for 'commandName', 'launchBrowser', 'applicationUrl', 'commandLineArgs', and 'environmentVariables'. The environment variables include ASPNETCORE_ENVIRONMENT, Portal__SupportEmail, and FeatureFlags__FromEnvironment.

```
1 {
2   "$schema": "https://json.schemastore.org/launchsettings.json",
3   "profiles": {
4     "CampusRouteLab": {
5       "commandName": "Project",
6       "launchBrowser": false,
7       "applicationUrl": "http://localhost:5187",
8       "commandLineArgs": "--Portal:Title=CampusHub.ConfigCenter.CLI --FeatureFlags:FromCommandLine=true",
9       "environmentVariables": {
10        "ASPNETCORE_ENVIRONMENT": "Development",
11        "Portal__SupportEmail": "config-support-env@campus.local",
12        "FeatureFlags__FromEnvironment": "true"
13      }
14    }
15  }
16 }
17
```

4. Демонстрация обязательных маршрутов

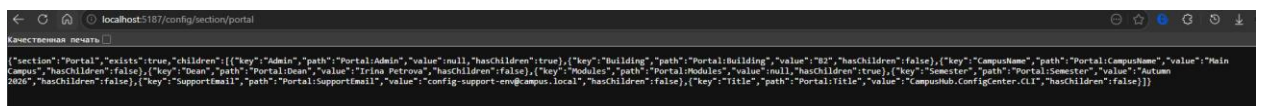
Все маршруты возвращают JSON и работают через IConfiguration или IOptions<T>.

/config/raw

A screenshot of a web browser showing the response of the /config/raw endpoint. The response is a JSON object containing environment variables and values for the application.

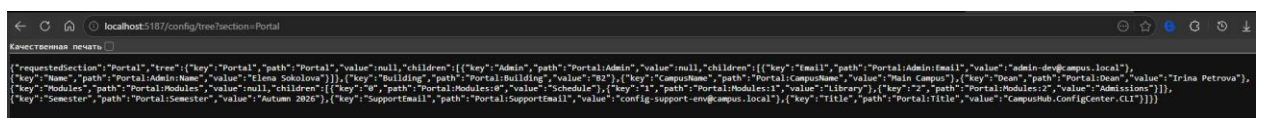
```
{ "environment": "Development", "values": { "Portal:Title": "CampusHub.ConfigCenter.CLI", "Portal:Semester": "Autumn 2026", "Portal:SupportEmail": "config-support-env@campus.local", "Notifications:Sender": "config-center-memory@campus.local", "FeatureFlags:EnableDiagnostics": "true" } }
```

/config/section/portal

A screenshot of a web browser showing the response of the /config/section/portal endpoint. The response is a JSON object containing configuration data for the portal section, including children and modules.

```
{ "section": "Portal", "exists": true, "children": { "key": "Admin", "path": "Portal:Admin", "value": null, "hasChildren": true }, { "key": "Building", "path": "Portal:Building", "value": "B2", "hasChildren": false }, { "key": "CampusName", "path": "Portal:CampusName", "value": "Main Campus", "hasChildren": false }, { "key": "Dean", "path": "Portal:Dean", "value": "Irina Petrova", "hasChildren": false }, { "key": "Modules", "path": "Portal:Modules", "value": null, "hasChildren": true }, { "key": "Semester", "path": "Portal:Semester", "value": "Autumn 2026", "hasChildren": false }, { "key": "SupportEmail", "path": "Portal:SupportEmail", "value": "config-support-env@campus.local", "hasChildren": false }, { "key": "Title", "path": "Portal:Title", "value": "CampusHub.ConfigCenter.CLI", "hasChildren": false } }
```

/config/tree?section=Portal

A screenshot of a web browser showing the response of the /config/tree?section=Portal endpoint. The response is a JSON object containing a tree structure of configuration data for the portal section.

```
{ "requestedSection": "Portal", "tree": { "key": "Portal", "path": "Portal", "value": null, "children": { "key": "Admin", "path": "Portal:Admin", "value": null, "children": { "key": "Email", "path": "Portal:Admin:Email", "value": "admin-dev@campus.local", "hasChildren": false }, "key": "Name", "path": "Portal:Admin:Name", "value": "Elena Sokolova", "hasChildren": false }, "key": "Building", "path": "Portal:Building", "value": "B2", "hasChildren": false }, "key": "CampusName", "path": "Portal:CampusName", "value": "Main Campus", "hasChildren": false }, "key": "Dean", "path": "Portal:Dean", "value": "Irina Petrova", "hasChildren": false }, "key": "Modules", "path": "Portal:Modules", "value": null, "children": { "key": "0", "path": "Portal:Modules:0", "value": "Schedule", "hasChildren": false }, "key": "1", "path": "Portal:Modules:1", "value": "Library", "hasChildren": false }, "key": "2", "path": "Portal:Modules:2", "value": "Admissions", "hasChildren": false }, "key": "Semester", "path": "Portal:Semester", "value": "Autumn 2026", "hasChildren": false }, "key": "SupportEmail", "path": "Portal:SupportEmail", "value": "config-support-env@campus.local", "hasChildren": false }, "key": "Title", "path": "Portal:Title", "value": "CampusHub.ConfigCenter.CLI", "hasChildren": false } }
```

/config/connection

Notifications:Sender	INI (config-center- ini@...), InMemory (config-center- memory@...)	InMemoryCollection	Добавлен после INI, имеет высший приоритет
----------------------	---	--------------------	--

7. Проекция на классы: разница между Get<T>() и IOptions<T>

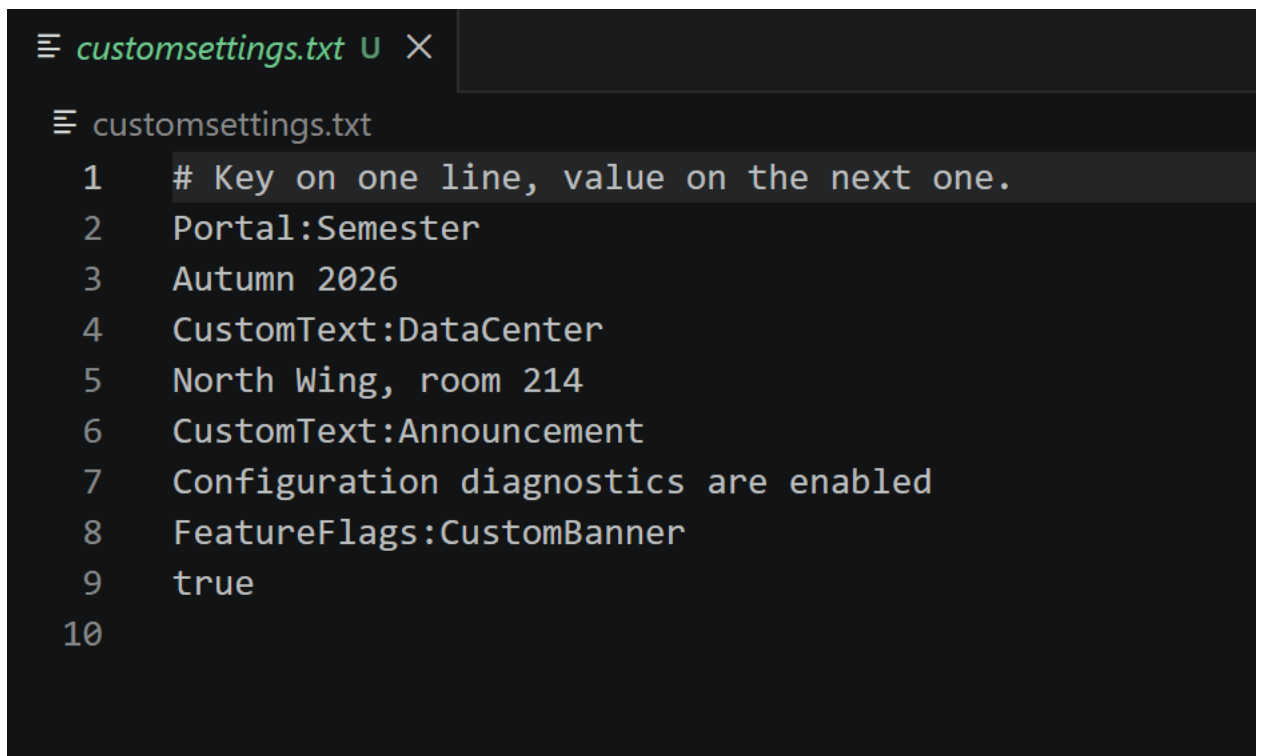
Характеристика	`Get<T>()` / `Bind()`	`IOptions<T>`
Место вызова	В коде endpoint или при старте	Внедряется через DI в конструктор
Жизненный цикл	Создаётся при каждом вызове (или один раз, если кэшировать вручную)	Синглтон (по умолчанию), создаётся один раз при старте
Интеграция с DI	Нет	Полная поддержка, валидация, post-configuration
Обновление при изменении файлов	Нет (читает состояние на момент вызова)	Нет (для динамического обновления нужны `IOptionsSnapshot` / `IOptionsMonitor`)
Применение в проекте	Используется в `~/config/bind` для демонстрации прямой привязки	Используется в `~/config/options` и в `PortalHeaderMiddleware`

8. Собственный провайдер конфигурации

Формат customsettings.txt:

Пары строк: нечётная строка = ключ, чётная строка = значение.

Строки, начинающиеся с #, и пустые строки игнорируются.



```
customsettings.txt
1  # Key on one line, value on the next one.
2  Portal:Semester
3  Autumn 2026
4  CustomText:DataCenter
5  North Wing, room 214
6  CustomText:Announcement
7  Configuration diagnostics are enabled
8  FeatureFlags:CustomBanner
9  true
10
```

Алгоритм работы TextConfigurationProvider:

1. TextConfigurationSource получает путь к файлу и передаёт его в Build().
2. Провайдер использует IFileProvider для чтения файла.
3. В методе Load() файл читается построчно, фильтруются комментарии и пробелы.
4. Проверяется чётность количества строк (иначе выбрасывается InvalidDataException).
5. Строки группируются по 2 и записываются в Dictionary<string, string?> с StringComparer.OrdinalIgnoreCase.
6. Словарь присваивается свойству Data, после чего конфигурационная система может запрашивать значения через стандартный API.

```
TextConfigurationProvider.cs U X
Configuration > C# TextConfigurationProvider.cs
7 {
18 public override void Load()
19 {
20     var fileInfo = _fileProvider.GetFileInfo(SourcePath);
21     if (!fileInfo.Exists)
22     {
23         throw new FileNotFoundException(
24             $"Не найден файл пользовательской конфигурации '{SourcePath}'.",
25             SourcePath);
26     }
27
28     using var stream = fileInfo.CreateReadStream();
29     using var reader = new StreamReader(stream);
30
31     var meaningfulLines = new List<string>();
32     while (!reader.EndOfStream)
33     {
34         var line = reader.ReadLine()?.Trim();
35         if (string.IsNullOrEmpty(line) || line.StartsWith('#'))
36         {
37             continue;
38         }
39
40         meaningfulLines.Add(line);
41     }
42
43     if (meaningfulLines.Count % 2 != 0)
44     {
45         throw new InvalidDataException(
46             $"Файл '{SourcePath}' должен содержать пары строк ключ/значение.");
47     }
48
49     var data = new Dictionary<string, string?>(StringComparer.OrdinalIgnoreCase);
50     for (var index = 0; index < meaningfulLines.Count; index += 2)
51     {
52         var key = meaningfulLines[index];
53         var value = meaningfulLines[index + 1];
54         data[key] = value;
55     }
56
57     Data = data;
58 }
```

9. Использование IOptions в Middleware

PortalHeaderMiddleware получает IOptions<PortalOptions> через DI:

```
17     public async Task InvokeAsync(HttpContext context)
18     {
19         var portal = _options.Value;
20
21         context.Response.Headers["X-Portal-Title"] = portal.Title;
22         context.Response.Headers["X-Portal-Semester"] = portal.Semester;
23
24         await _next(context);
25     }
```

Middleware добавляет заголовки ко **всем** HTTP-ответам приложения до передачи управления следующему обработчику.

10. Заключение

В ходе работы была реализована полноценная система диагностики конфигурации ASP.NET Core. Применены все обязательные типы провайдеров (JSON, XML, INI, Env, CLI, Memory, Custom), продемонстрирован механизм приоритетов при конфликтах ключей, создана привязка к сложным объектам и настроена передача настроек в middleware через `IOptions<T>`. Код избегает хардкода, полностью опирается на `IConfiguration` и готов к расширению.